

软件安全第九次实验报告

姓名：林晖鹏 学号：2312966

一、实验名称：

Angr应用示例

二、实验要求：

根据课本8.4.3章节，复现 sym-write 示例的两种f Angr求解方法，并就如何使用 angr 及如何解决一些实际问题做一些探讨。

三、实验过程：

1. 安装Python3和Angr

我的电脑中本身有Python3了，这里下载流程就不赘述了，这里放一个python下载的网址：
<https://www.python.org/downloads/>

然后我们在终端中输入指令用**pip**下载**Angr**即可：（这里需要配一些依赖库，一般会自动下载缺失的依赖库）

代码块

```
1 pip3 install Angr
```

接下来我们打开一个py文件，导入Angr，然后尝试打印版本信息。从图中我们可以看到成功导入Angr并打印信息，说明安装成功。

```
lab9.py
1  import angr
2  print(angr.__version__)

问题  输出  调试控制台  终端  端口  评论

PS C:\Users\lhppp\Desktop\未命名文件夹> & C:/Users/lh
n/Python313/python.exe c:/Users/lhppp/Desktop/未命名文
9.2.154
```

2. 复现求解方法一

我们可以查阅github上的官方文档，里面有相关代码。这里放链接：
<https://github.com/angr/angr-doc/tree/master/examples/sym-write>

2.1 创建待符号执行的代码文件

创建一个文件 `issue.c` ,写入下面的代码，并编译文件。

代码块

```
1  #include <stdio.h>
2
3  char u = 0;
4  int main(void)
5  {
6      int i, bits[2] = {0, 0};
7      for (i = 0; i < 8; i++)
8      {
9          bits[(u & (1 << i)) != 0]++;
10     }
11     if (bits[0] == bits[1])
12     {
13         printf("you win!");
14     }
15     else
16     {
17         printf("you lose!");
18     }
19     return 0;
20 }
```

小问题：在编译执行solve的时候，发现它案例代码里面的地址是linux编译下文件对应的地址，所以如果在mac或者win下编译文件，会导致solve.py找不到这个u变量。

我尝试了去用angr本身去寻找u地址变量，但是求解出来还是不对。也尝试了用各种编译软件如gdb这些去查找变量的地址，但是在带入代码之后都不对。

代码块

```
1  # 获取 u 的地址
2  u_symbol = p.loader.find_symbol('u')
3  if u_symbol:
4      u_addr = u_symbol.rebased_addr
5      print(f"找到符号 'u' 的地址: 0x{u_addr:x}")
6  else:
7      print("找不到符号 'u', 尝试备选方法")
8      data_section = p.loader.main_object.sections_map['.data']
9      if data_section:
10         u_addr = data_section.vaddr + 0x30
11         print(f"通过 .data 节计算 u 的地址: 0x{u_addr:x}")
12     else:
13         print("无法确定 u 的地址")
14     return []
```

所以下面的复现过程我都是用github上面的linux环境下编译后的二进制文件来复现。

2.2 创建Angr求解代码

创建一个文件叫 `solve.py`，放入下面的代码：

代码块

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  """
5  Author: xoreaxeaxeax
6  Modified by David Manouchehri <manouchehri@protonmail.com>
7  Original at https://lists.cs.ucsb.edu/pipermail/angr/2016-August/000167.html
8
9  The purpose of this example is to show how to use symbolic write addresses.
10 """
11
12 import angr
13 import claripy
14
15 def main():
16     p = angr.Project('./issue', load_options={"auto_load_libs": False})
```

```

17
18     # By default, all symbolic write indices are concretized.
19     state = p.factory.entry_state(add_options=
{angr.options.SYMBOLIC_WRITE_ADDRESSES})
20
21     u = claripy.BVS("u", 8)
22     state.memory.store(0x804a021, u)
23
24     sm = p.factory.simulation_manager(state)
25
26     def correct(state):
27         try:
28             return b'win' in state.posix.dumps(1)
29         except:
30             return False
31     def wrong(state):
32         try:
33             return b'lose' in state.posix.dumps(1)
34         except:
35             return False
36
37     sm.explore(find=correct, avoid=wrong)
38
39     # Alternatively, you can hardcode the addresses.
40     # sm.explore(find=0x80484e3, avoid=0x80484f5)
41
42     return sm.found[0].solver.eval_upto(u, 256)
43
44 def test():
45     good = set()
46     for u in range(256):
47         bits = [0, 0]
48         for i in range(8):
49             bits[u & (1 << i) != 0] += 1
50         if bits[0] == bits[1]:
51             good.add(u)
52
53     res = main()
54     print(set(res) == good)
55
56 if __name__ == '__main__':
57     print(repr(main()))
58     test()
59

```

运行代码之后得到结果：

代码块

```
1 [51, 57, 60, 240, 75, 139, 78, 197, 23, 142, 90, 29, 209, 154, 212, 99, 163,
  102, 108, 166, 172, 105, 169, 114, 53, 120, 225, 184, 178, 71, 135, 77, 83,
  89, 141, 147, 153, 92, 86, 150, 156, 202, 101, 106, 165, 43, 226, 113, 46,
  177, 116, 232, 180, 58,
2 198, 15, 201, 195, 85, 204, 30, 149, 210, 27, 216, 39, 45, 170, 228, 54]
```

```
WARNING | 2025-05-08 12:45:42,431 | angr.storage.memory_mixins.default_filler_mixin
| Filling register ebx with 4 unconstrained bytes referenced from 0x8048523 (__libc
_csu_init+0x3 in issue (0x8048523))
[51, 57, 60, 240, 75, 139, 78, 197, 23, 142, 90, 29, 209, 154, 212, 99, 163, 102, 10
8, 166, 172, 105, 169, 114, 53, 120, 225, 184, 178, 71, 135, 77, 83, 89, 141, 147, 1
53, 92, 86, 150, 156, 202, 101, 106, 165, 43, 226, 113, 46, 177, 116, 232, 180, 58,
198, 15, 201, 195, 85, 204, 30, 149, 210, 27, 216, 39, 45, 170, 228, 54]
PS C:\Users\lhppp\Desktop\lab9> □
```

代码中还有一个test函数，用来检测正确性，但是原代码没有调用，在调用之后返回了正确性：

```
| Filling register ebx with 4 unconstrained byte
_csu_init+0x3 in issue (0x8048523))
True
```

2.3 代码分析

在上述Angr示例中，几个关键步骤如下：

- 创建一个Angr工程，并且载入二进制文件。
- 这里需要注意要把auto_load_libs设置为false，不自动载入依赖的库，否则可能会给符号执行带来不必要的麻烦。
- 使用函数entry_state()初始化一个模拟程序状态的SimState对象state。
- 将要求解的变量u符号化，注意这里符号化后的变量存在二进制文件的存储区。（这里注意变量的地址要正确，否则求解答案为空）
- 创建模拟管理器sm，进行程序执行管理。初始化的state可以经过模拟执行得到一系列的states，模拟管理器sm的作用就是对这些states进行管理。
- 进行符号执行得到想要的状态，得到想要的状态。上述程序所表达的状态就是，符号执行后，源程序里打印出的字符串里包含win字符串，而没有包含lose字符串。在这里，状态被定义为两个函数，通过符号执行得到的输出state.posix.dumps(1)中是否包含win或者lose的字符串来完成定义。
- 获得到state之后，通过solver求解器，求解u的值。

3. 复现求解方法二

3.1 创建Angr求解代码

创建一个文件叫 `solve.py`，放入下面的代码：

代码块

```
1  #!/usr/bin/env python
2  # coding=utf-8
3  import angr
4  import claripy
5  def hook_demo(state):
6      state.regs.eax = 0
7
8  p = angr.Project("./issue", load_options={"auto_load_libs": False})
9  # hook 函数: addr 为待 hook 的地址
10 # hook 为 hook 的处理函数, 在执行到 addr 时, 会执行这个函数, 同时把当前的 state 对象作为参数传递过去
11 # length 为待 hook 指令的长度, 在执行完 hook 函数以后, angr 需要根据 length 来跳过这条指令, 执行下一条指令
12 # hook 0x08048485 处的指令 (xor eax,eax), 等价于将 eax 设置为 0
13 # hook 并不会改变函数逻辑, 只是更换实现方式, 提升符号执行速度
14 p.hook(addr=0x08048485, hook=hook_demo, length=2)
15 state = p.factory.blank_state(addr=0x0804846B,
16 add_options={"SYMBOLIC_WRITE_ADDRESSES"})
17 u = claripy.BVS("u", 8)
18 state.memory.store(0x0804A021, u)
19 sm = p.factory.simulation_manager(state)
20 sm.explore(find=0x080484DB)
21 st = sm.found[0]
22 print(repr(st.solver.eval(u)))
```

```
| Filling memory at 0x7fff0000 with 4 unconstrained bytes referenced from 0x8048472
(main+0x7 in issue (0x8048472))
WARNING | 2025-05-08 12:42:28,549 | angr.storage.memory_mixins.default_filler_mixin
| Filling register ebp with 4 unconstrained bytes referenced from 0x8048475 (main+0
xa in issue (0x8048475))
226
PS C:\Users\lhnpp\Desktop\lab9> |
```

执行得到结果226。

3.2 代码分析

第二种方法只输出了解，而不是第一种方法那样输出一大堆解，两个方法还有有比较多区别的。

第二种解法与前一种解法的区别在于以下三处：

- a. 采用了hook函数，将0x08048485处的长度为2的指令通过自定义的hook_demo进行替代，功能是一致的，原始xoreax,eax和state.regs.eax=0是相同的作用，这里只是演示，可以将一些复杂的系统函数调用，比如printf等，可以进行hook，提升符号执行的性能。
- b. 进行符号执行得到想要的状态，有变化，变更为find=0x080484DB。因为源程序win和lose是互斥的，所以，只需要给定一个find条件即可。
- c. 最后，eval(u)替代了原来的eval_upto，将打印一个结果出来。所以上面的第二种解法中，只输出了一个结果，而不是像第一个解法那样输入一大堆的解。

4. angr使用方法

angr 提供了多种分析功能：

- **符号执行**：探索程序路径，识别输入与输出之间的约束关系；
- **CFG 构建**：还原程序结构，辅助静态分析；
- **污点分析**：追踪敏感数据如何在程序中传播；
- **程序约束求解**：自动生成满足特定条件的输入；
- **自动化漏洞检测与利用**

angr主体的思想是，利用符号执行来探索程序路径，让angr去探索路径，直到输出目标输出，这样就能得到特定的输入。

下面给出一个使用的基本框架代码：

代码块

```
1  import angr
2
3  project = angr.Project("path/to/binary", auto_load_libs=False)
4
5  # 设置起始地址
6  state = project.factory.entry_state()
7
8  # 构建模拟管理器
9  simgr = project.factory.simulation_manager(state)
10
11 # 进行符号执行，直到找到我们想要的目标状态
12 simgr.explore(find=0xdeadbeef, avoid=0xdeadfa11)
13
14 if simgr.found:
15     found_state = simgr.found[0]
16     print(found_state.posix.stdin.concrete)
17
```

利用angr去自动探索路径，我们能拿来干下面这些事情：

1. 输入验证破解（CTF）

比如程序让你输入一个正确的密码，你就可以用angr去自动探索，直到得到正确的输出，此时得到正确的密码。

代码块

```
1  if (strcmp(input, "supersecret123") == 0) {
2      puts("Correct!");
3  } else {
4      puts("Wrong!");
5  }
```

2. 漏洞触发路径

在比如程序有一个漏洞，你可以用angr去自动探索，知道哪个输入可以去触发这个漏洞（干坏事）。

可以利用 路径约束系统 + 符号执行 模拟程序执行，寻找能让程序执行到该漏洞代码段（如某地址）的输入条件。

3. 自动 Exploit 构造

可以配合 内存建模与路径约束分析，验证是否能构造出输入使栈中布置满足 payload 条件（比如寄存器/内存具体值）。也可结合 ROP 模块（`project.analyses.ROP()`）自动分析程序中可用的 gadget。

这样可以构造一个精确的 Exploit Payload，比如覆盖返回地址、填充特定寄存器、触发 ROP。

4. CFG 静态结构分析

我们还可以使用 `CFGFast()` 或 `CFGAccurate()` 快速构建控制流图，找出所有函数、调用点、跳转表等信息。来获取程序的控制流图、函数调用结构、调用关系图（类似于 IDA、Ghidra 中的功能），但用 angr 自动化分析。

除了上面这些用法，还有很多可以用来（干坏事）干其他事情的地方。

四、实验总结

- 在本次实验中，我们复现了两个使用angr进行符号执行求解输入的例子。了解并收悉了angr的用法和思想。
 - 也探讨了angr的更多应用方式，并学习了angr的基本使用框架
-